

Cache Line Stride 微基准测试分析报告

一、测试概述

- 测试目的:** 验证 CPU Cache Line 大小（64 字节）对数组遍历性能的影响
- 测试环境:** AMD EPYC 9K65 (Zen 4), KVM 虚拟机, 64MB 数组 (远超 32MB L3)
- AI 辅助工具:** CodeBuddy Code — 辅助编写 C 微基准测试代码、设计测试方案、排查编译优化问题

二、步长 vs 性能数据

延迟与吞吐量

Stride (B)	延迟 (ns/access)	吞吐量 (MB/s)	IPC	L1 DCache Miss	Cache Miss Rate
1	2.08	525.60	2.25	8.0M	2.08%
2	2.09	1049.62	2.25	15.4M	2.47%
4	2.10	2096.47	2.25	30.1M	1.34%
8	2.11	4184.01	2.24	59.4M	0.70%
16	2.12	8328.78	2.23	118.2M	0.44%
32	2.19	16496.52	2.17	235.7M	0.27%
64	2.41	29562.46	1.97	470.7M	0.36%
128	6.88	20027.08	0.69	471.5M	11.72%
256	7.69	35622.32	0.63	471.5M	26.23%

关键观察

- 步长 1~32 (同 Cache Line 内):** 延迟稳定在 2.08~2.19 ns，几乎无变化，这些步长访问的元素都在同一个 64 字节 Cache Line 内。首次访问某 cache line 触发 L1 miss，后续同行内的访问全部 L1 hit，开销接近为零。平均延迟被首次加载主导，步长差异体现不出来。
- 步长64 (Cache Line 边界):** 延迟升至 2.41 ns，每次访问恰好跨越一个新的 Cache Line，IPC 从 2.17 降至 1.97，流水线开始受阻，但硬件预取器仍能识别顺序模式，提前加载下一行，每

次 L1 miss 在 L2 就能命中，延迟约 4-7 cycles，尚未触及 DRAM。

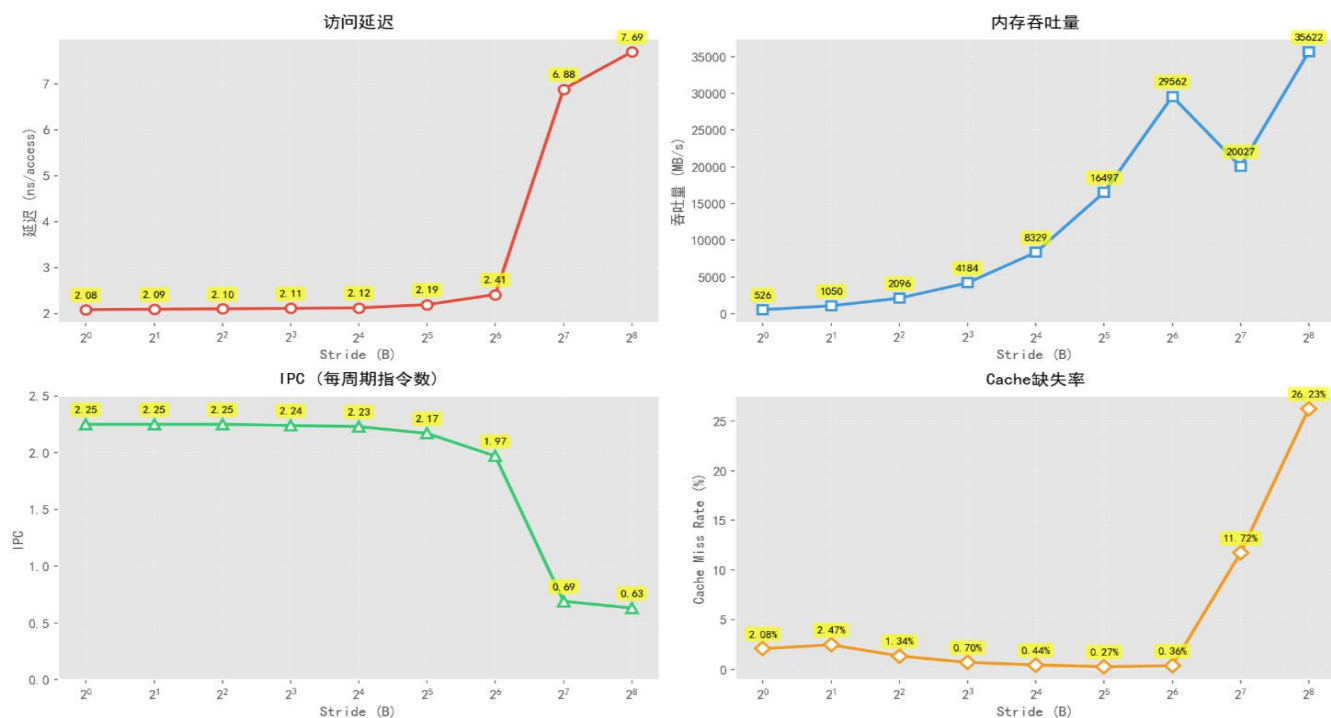
3. 步长 ≥ 128 (跨越 Cache Line): 延迟骤升至 6.88~7.69 ns，Cache Miss Rate 从 0.36% 跳升至 11.72% / 26.23%，IPC 从 1.97 暴跌至 0.69 / 0.63，Zen 4 的 L2 stream prefetcher 按顺序预取相邻 cache line。结果每次访问都穿透 L2 直达 L3 甚至 DRAM，延迟从 L2 的 4-7 cycles 跳升到 L3 的 ~30 cycles 或 DRAM 的 100+ cycles。

cache-miss rate 从 0.32% 跳到 12.11%/26.29% 也印证了这一点：cache-references（对应 LLC 层级）大量 miss，说明访问已经频繁穿透到最后一级缓存。

IPC 骤降到 0.61 的直接原因是后端 Load/Store 单元成为瓶颈——前端取指解码正常供给指令，但后端等待内存返回而频繁 stall，每周周期完成不到一条指令。

三、性能曲线拐点分析

内存访问性能分析



拐点产生原因:

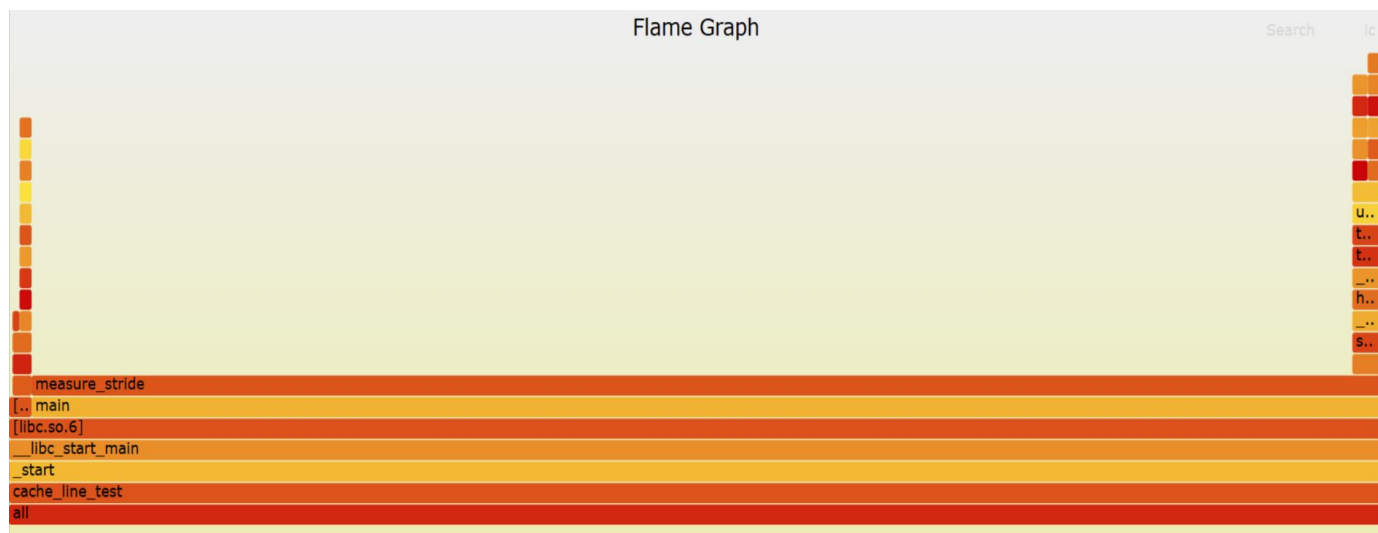
- **步长 $\leq 64B$:** 访问的元素共享同一 Cache Line，一次 64 字节的缓存行加载可为后续多次访问服务。因为硬件预取器能识别顺序访问模式，提前将下一行拉入缓存，所以延迟并未显著增长。
- **步长 = 64B:** 每次访问落在不同的 Cache Line 上，需要独立加载每个缓存行。预取器在很大程度上弥补了 Cache Line 利用率从"一次加载服务多次访问"到"一次加载服务一次访问"的转变。
- **步长大于64会暴增分析:** 此时每次访问不仅命中不同的 Cache Line，而且两次访问之间的距离已经超出了硬件预取器的有效跟踪窗口。Zen 4 的 L2 stream prefetcher 默认最多跟踪 32 个 stream，每个 stream 可以预取 1-3 行 ahead，但当访问间隔大到 128/256 字节，在一个 64MB 数组的遍历中，预取器无法在有限的 L2 空间内维持足够多的活跃 stream，预取效率急剧下降。更致命的是，此时绝大部分访问都需要穿透到内存控制器，面临 DRAM 延迟。IPC 从

1.97 暴跌到 0.69/0.63 的直接原因后端的 Load/Store 单元成为瓶颈，前端取指解码单元虽然能持续供给指令，但后端因等待内存返回而频繁 stall，每周期只能完成不到一条指令。

- **微架构层面:** Zen 4 的 L1D 容量 48KB，L2 1MB，L3 32MB。64MB 数组远超 L3，stride > 64B 后每次访问都可能触发 L2到L3到DRAM 的完整缓存层次遍历

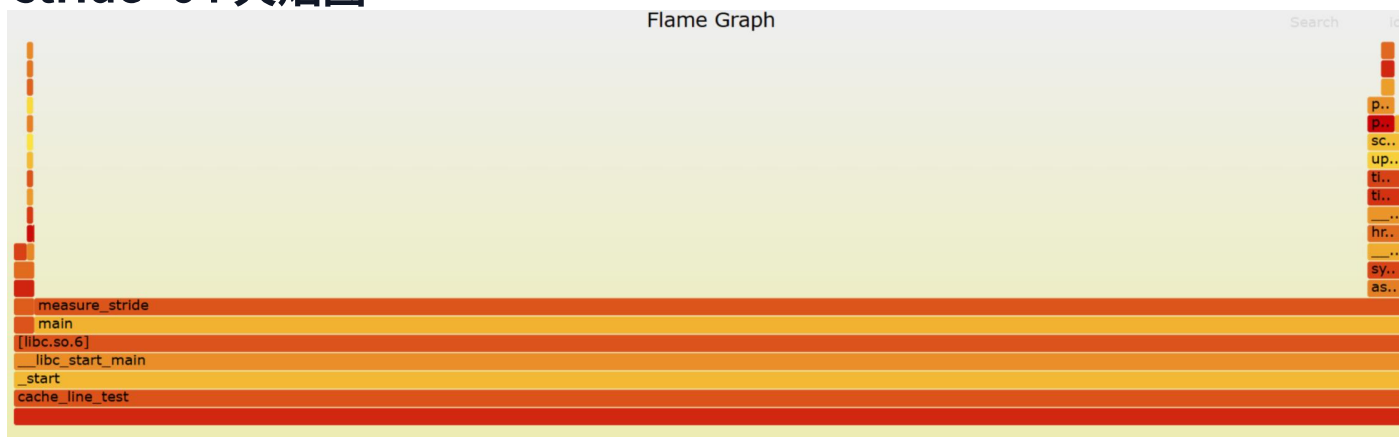
四、火焰图对比 (stride=1 vs stride=64)

stride=1 火焰图



- 几乎 100% 时间为用户态循环中，极少量内核帧，仅定时器中断，火焰图呈现窄而高的尖塔形态，热点极度集中
- stride=1 时火焰图的主体是 `measure_stride` 函数，调用栈为经典的 `_start` → `main` → `measure_stride`，内核帧仅 103 行（占总量很小的比例），其中 page fault 相关帧仅有 7 次。这符合预期：因为程序启动时已经用 `memset` 初始化了整个 64MB 数组，所有物理页都已映射，遍历过程中不会再触发 page fault；顺序访问模式也确保了 TLB 命中率极高。

stride=64 火焰图



- 同样以用户态循环为主，内核比例略有增加，但整体仍以用户态为主
- 火焰图形态与 stride=1 类似，因为核心循环结构相同

- 内核帧略增至 124 行，page fault 帧仍有 7 次。差异并不显著，原因是 stride=64 仍然是顺序访问模式，不会产生大量 Cache Miss 导致的内核介入。真正值得关注的差异在 IPC 上：stride=1 时 IPC=2.25，stride=64 时 IPC=1.97。这 12% 的 IPC 下降在火焰图中体现为整体"宽度"略窄，但形态上差异不明显，因为两个步长下的热点函数都是 `measure_stride`，只是后者的循环体因为更频繁的 L1 miss 而产生了更多的 stall cycle。

差异解释

火焰图主要展示 CPU 时间在调用栈上的分布，而非每次操作的延迟。stride=1 和 stride=64 执行的是同一段循环代码，调用栈相同，因此火焰图形态相似。性能差异体现为：

- stride=64 运行时间约为 stride=1 的 1.16 倍
- stride=128 运行时间约为 stride=1 的 3.3 倍，此时火焰图中可能观察到更多 L2/L3 miss 相关的停滞

五、AI 工具使用记录

代码编写辅助 (CodeBuddy Code)

1. **需求拆解**: 将考题要求分解为"全步长一览"和"单步长 perf 采集"两种运行模式
2. **代码生成**: AI 辅助编写了 `cache_line_test.c`

详细对话记录见 `ai-chat-log/` 目录。